[CSED233-01] Data Structure

B Tree

Seung-Hwan Baek

B

Balanced Search Trees

- Memory-based search trees
  - Balanced BST
    - AVL (Adelson-Velskii & Landis) trees
    - Red-black trees
    - Splay trees, …
  - Balanced multi-way search tree
    - 2-3, 2-3-4 trees (B-trees)

- Disk-based search trees
  - Balanced multi-way search trees
    - B-trees (B+, B*)
    - Prefix B-trees

B-tree (of order m)

- m-way search tree satisfying:

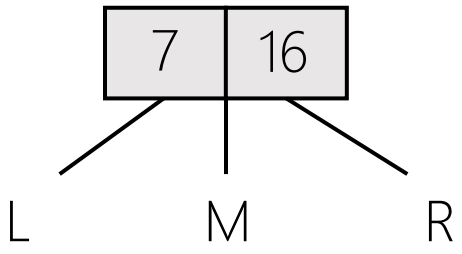
Shape Property

- Root has at least 2 children
- All internal nodes except root have at least $\lceil m/2 \rceil$ children
- All leaf nodes are at the same level

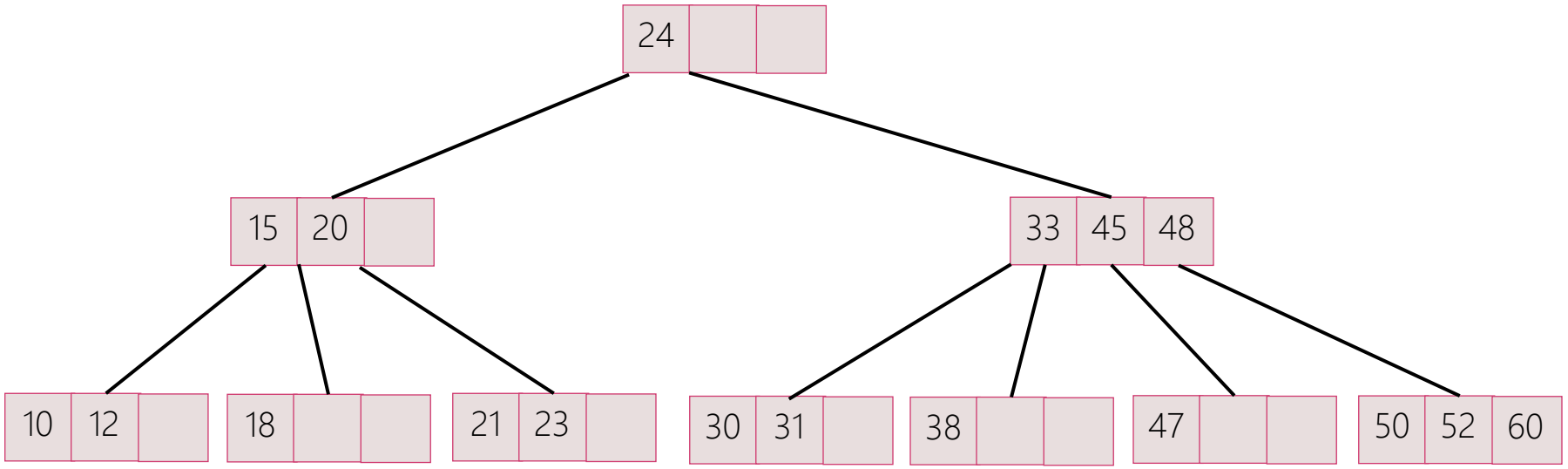
Search Tree Property

Example: order 3

- All keys in left subtree are smaller than first key
- All keys in middle subtree are greater than or equal to first key, and smaller than second key
- All keys in right subtree are greater than or equal to second key



B-tree: Example



- B-tree of order 4
  - Search tree property + Shape property
- Height Bound = $O(\log n)$

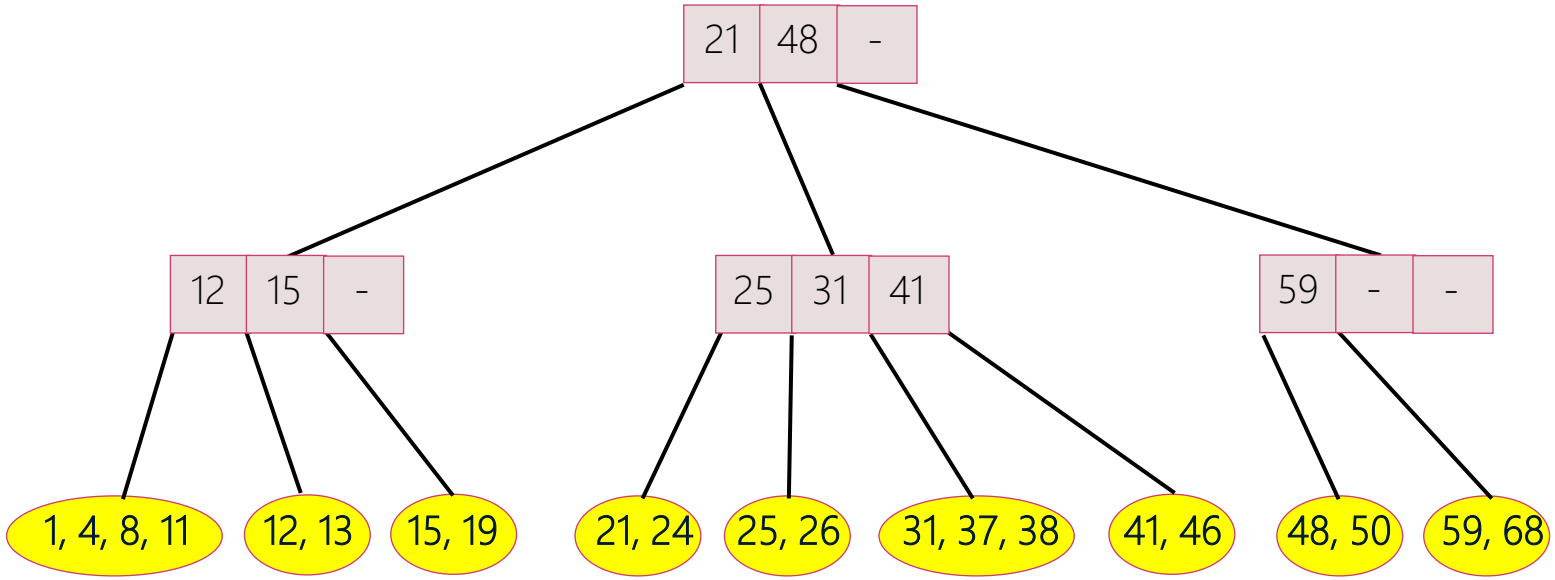
B-tree

- Analysis
  - Insert, Delete, Search: $O(\log n)$
    - where the log base is the average branching factor of the tree

- Implementation of B-tree node
  - Normally equivalent to a disk block
    - A block is the largest contiguous amount of disk space that can be allocated to a file
  - Typically allows 100 or more children (more than 2-3)
    - B-tree and its variants are extremely shallow

- Many keys in each internal node → the height reduced → fewer node accesses
  - Critical when data is stored in secondary places such as disk

B$^+$-tree (of order 4)



- Actual records are stored only at the leaf nodes
  - A leaf node may store more or less than m (generally, between $\lceil m/2 \rceil$ & m) actual records
- Easier to store more keys in internal nodes → shallower tree → fewer node accesses

B tree vs. B+ Tree

- Data is in every node vs. leaf nodes
- Duplicate keys not allowed vs. allowed

- B* tree
  - B^+-tree variant
  - More densely packed
  - Each node is at least 2/3 full

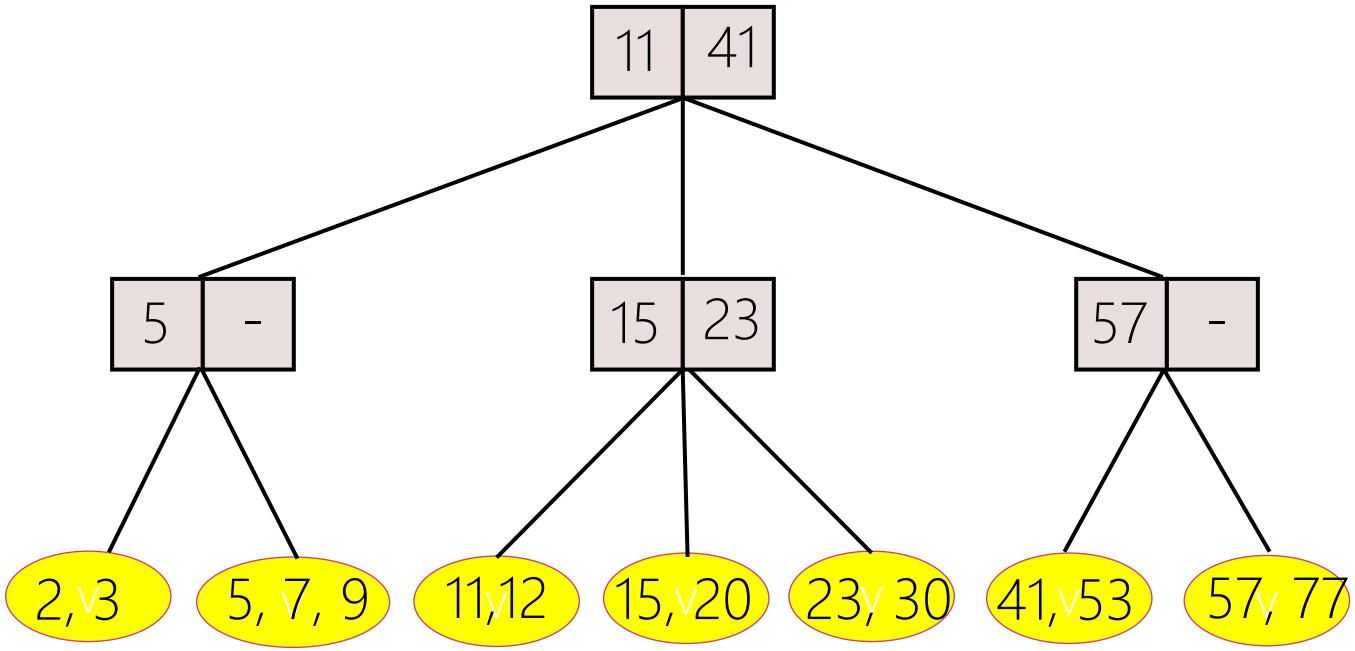
Quiz. B-tree vs. B+-tree

A B-tree can locate a key at an internal node and return early, while a B+-tree always descends to a leaf. So B-tree search seems faster in the best case.

Yet virtually every modern database uses B+-trees as its primary index. Why?

B+ tree of Order 3

- Actual records are in the *leaves*
  - B⁺-tree of order 3



- The keys are in the leaves
  - linearly ordered from left to right

2-3 Tree: Shape Property

- A tree with the following *shape properties*:
  - Each internal node has 2 or 3 children
  - All leaves are at the same level (→ Perfectly height-balanced)

- Internal nodes
  - Store one or two key values
  - Placeholders to guide the search

- Leaf nodes
  - All *actual records* - (*key*, *info*) pairs are stored at the leaves

- Alternative structure
  - Actual records are stored in both leaves and internal nodes
    - B-tree of order 3

Quiz

- h: a 2-3 tree's height
- n: the # of leaves in a 2-3 tree
- What are the minimum and maximum values of n?

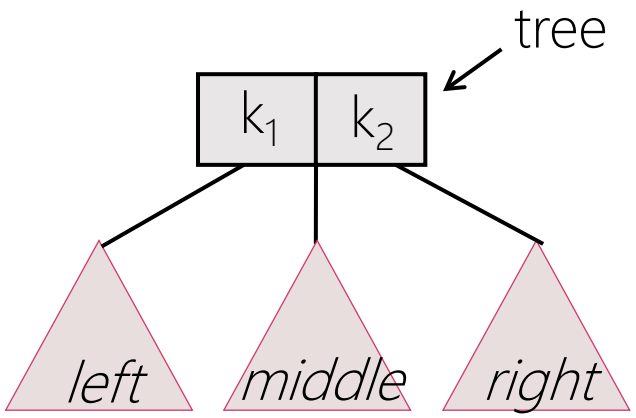
Quiz. Why 2–3 and not 1–4?

A 2-3 tree guarantees $O(\log n)$ search because every internal node has 2 or 3 children and all leaves sit at the same level.

Suppose we relaxed the rule to allow 1 to 4 children per node. Would search still be $O(\log n)$? If not, which part of the analysis breaks — and which specific node configuration causes the failure?

2-3 Tree: Search

- $search(x, tree)$
  - If $x < k_1$, $search(x, left_subtree)$
  - If $k_1 < x < k_2$, $search(x, middle_subtree)$
  - If $x > k_2$, $search(x, right_subtree)$
  - If $x = k_1 \; or \; k_2$
    - then the search key x is in the tree



- Search by simply moving down the tree

- Time complexity = ?

Quiz

A 2-3 tree has shorter height than a BST, so search is faster — right?

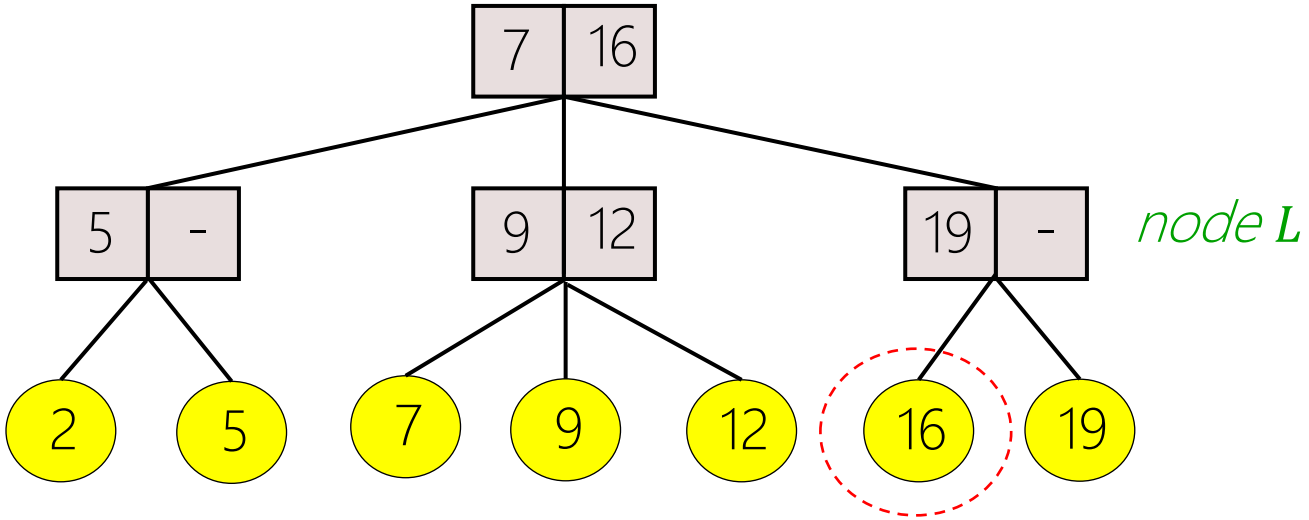
2-3 Tree: Insert

- Find a node L
  - One of whose children would contain the key if it were in the tree
- If L is not full
  - Then the new key is added
- If L is already full
  - Split L into two nodes L and L' (dividing the keys evenly among the two nodes)
  - Promote a copy of the *least-valued key* in L'

- Promotion may cause the parent to split in turn
  - Perhaps eventually leading to splitting the root (causing the 2-3 tree to gain a new level)

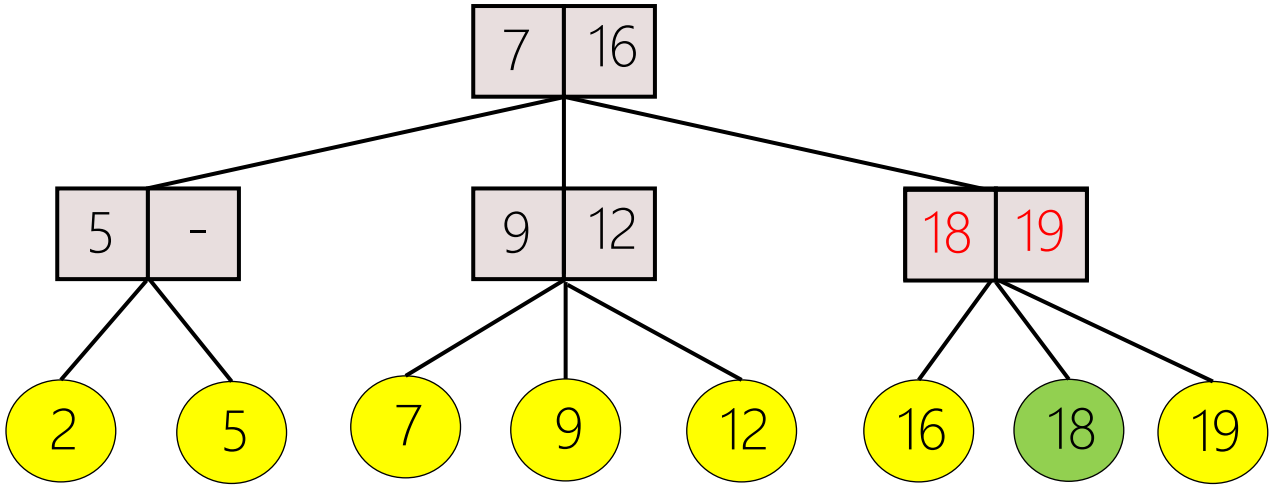
2-3 Tree: Insert - Example

- Insert a record with key value 18
  - Find the node L, one of whose children would contain 18 if it were in the tree
  - Since the parent L is not full, the key 18 can be added with no further modification to the tree



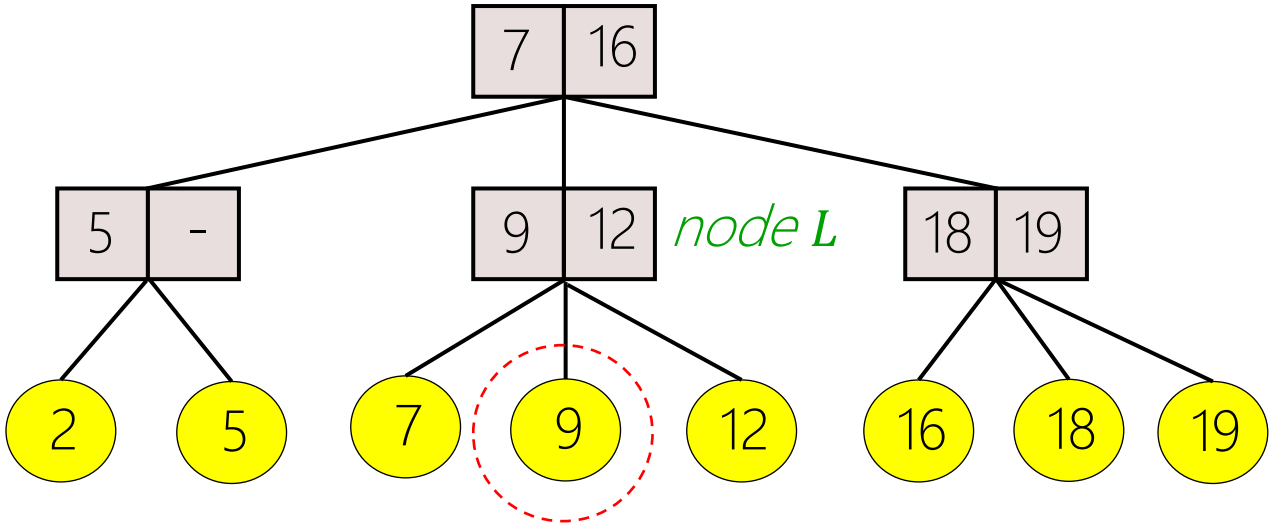
2-3 Tree: Insert - Example

- Insert a record with key value 18
  - Find the node L, one of whose children would contain 18 if it were in the tree
  - Since the parent L is not full, the key 18 can be added with no further modification to the tree
  - Update the key values in the parents



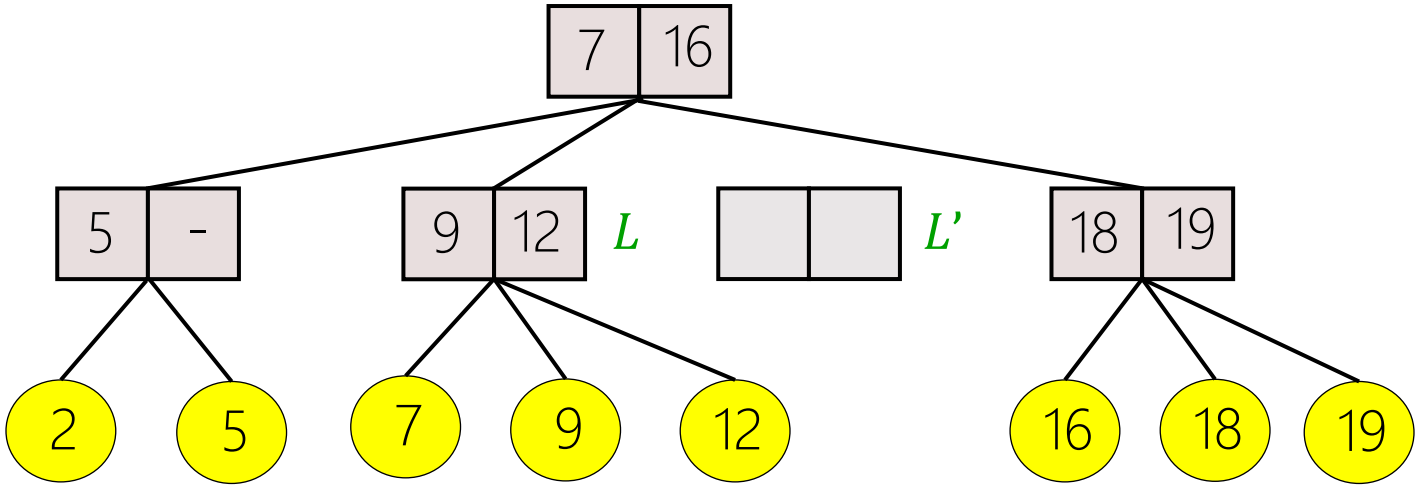
2-3 Tree: Insert - Example

- Insert a record with key value 10
  - Find the node L, one of whose children would contain 10 if it were in the tree
  - Since the parent L is already full, it has to be split (into L & L')



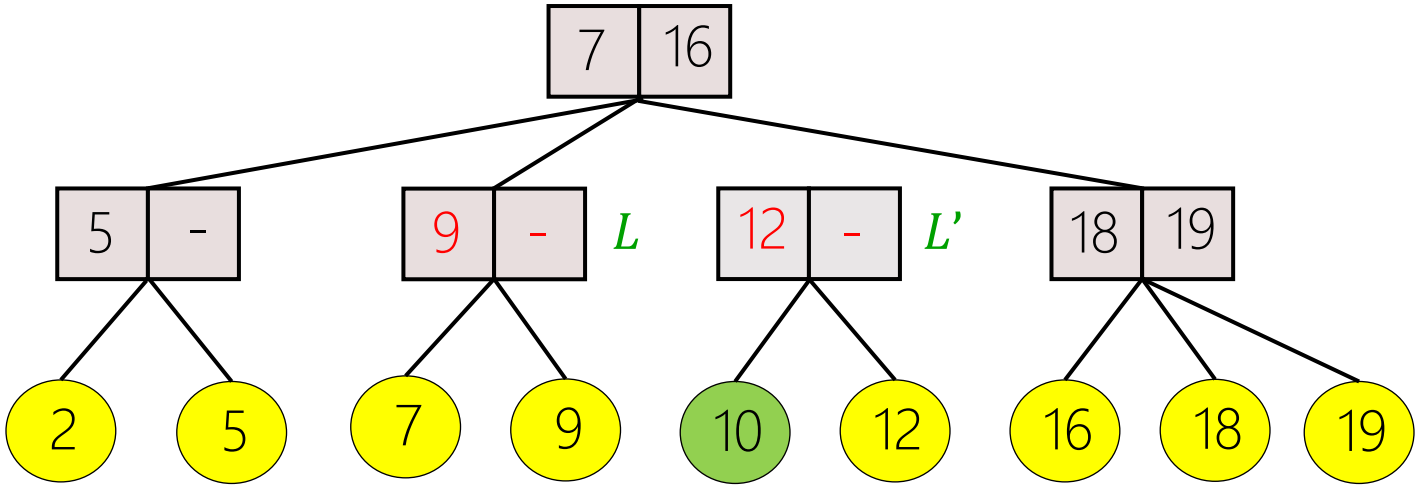
2-3 Tree: Insert - Example

- Insert a record with key value 10
  - Find the node L, one of whose children would contain 10 if it were in the tree
  - Since the parent L is already full, it has to be split (into L & L')
    - Next rearrange the keys evenly among the two nodes
    - Update the key values in the parents



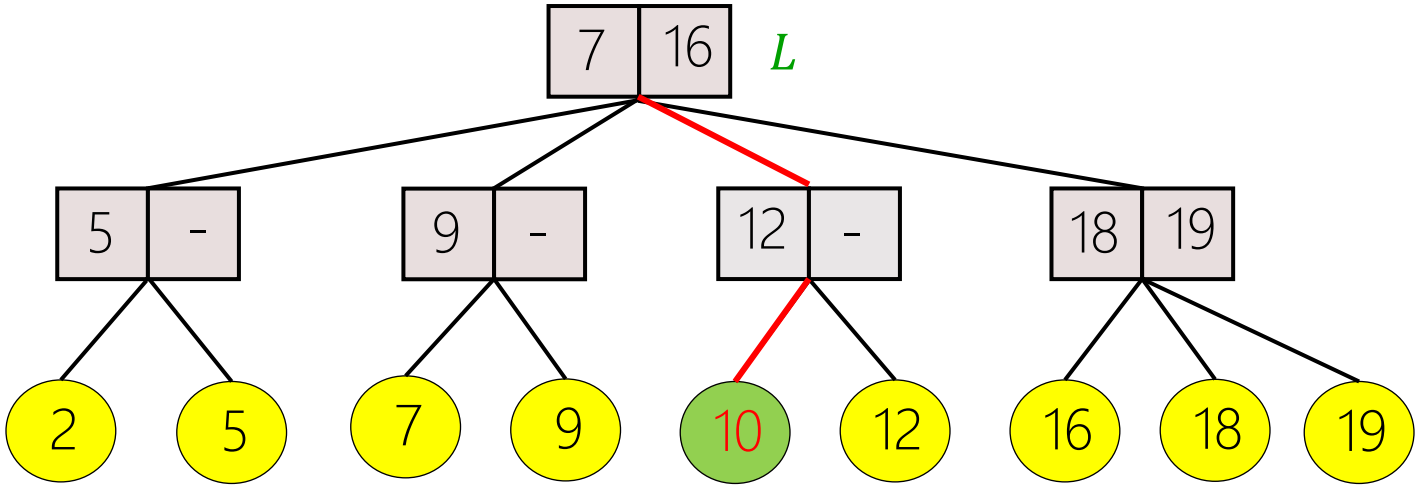
2-3 Tree: Insert - Example

- Insert a record with key value 10
  - Find the node L, one of whose children would contain 10 if it were in the tree
  - Since the parent L is already full, it has to be split (into L & L')
    - Next rearrange the keys evenly among the two nodes
    - Update the key values in the parents
  - Promote a copy of the *least-valued key 10* in L'



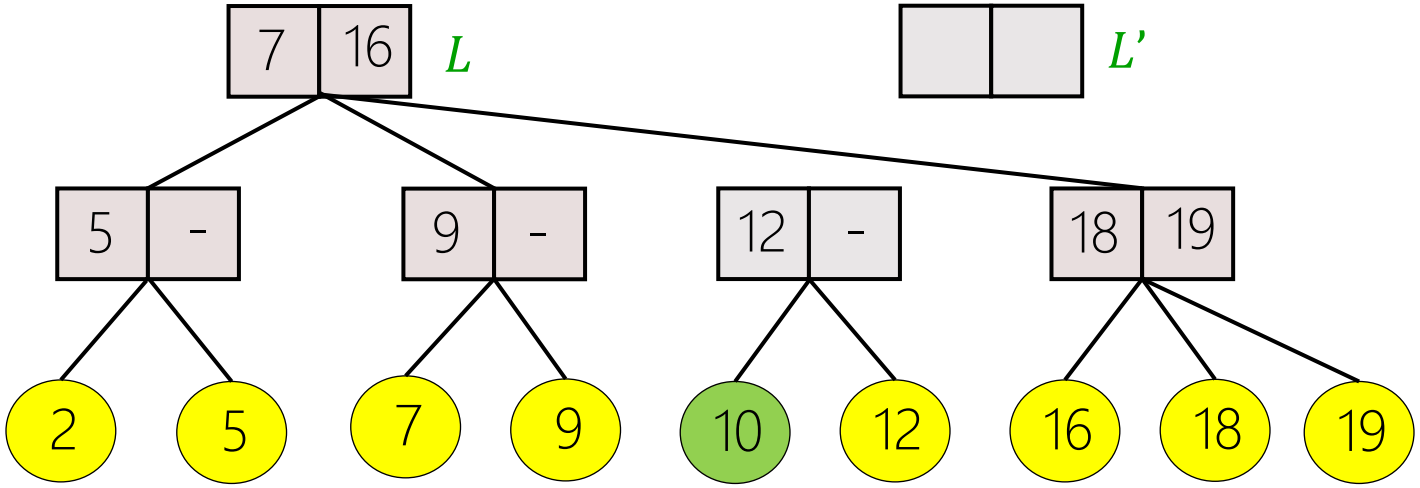
2-3 Tree: Insert - Example

- Insert a record with key value 10
  - Promote a copy of the *least-valued key 10* in L'
  - Grandparent is already full
    - So the *promoted key 10* cannot be inserted
  - Split-and-promote process again
    - Split L into L & L'



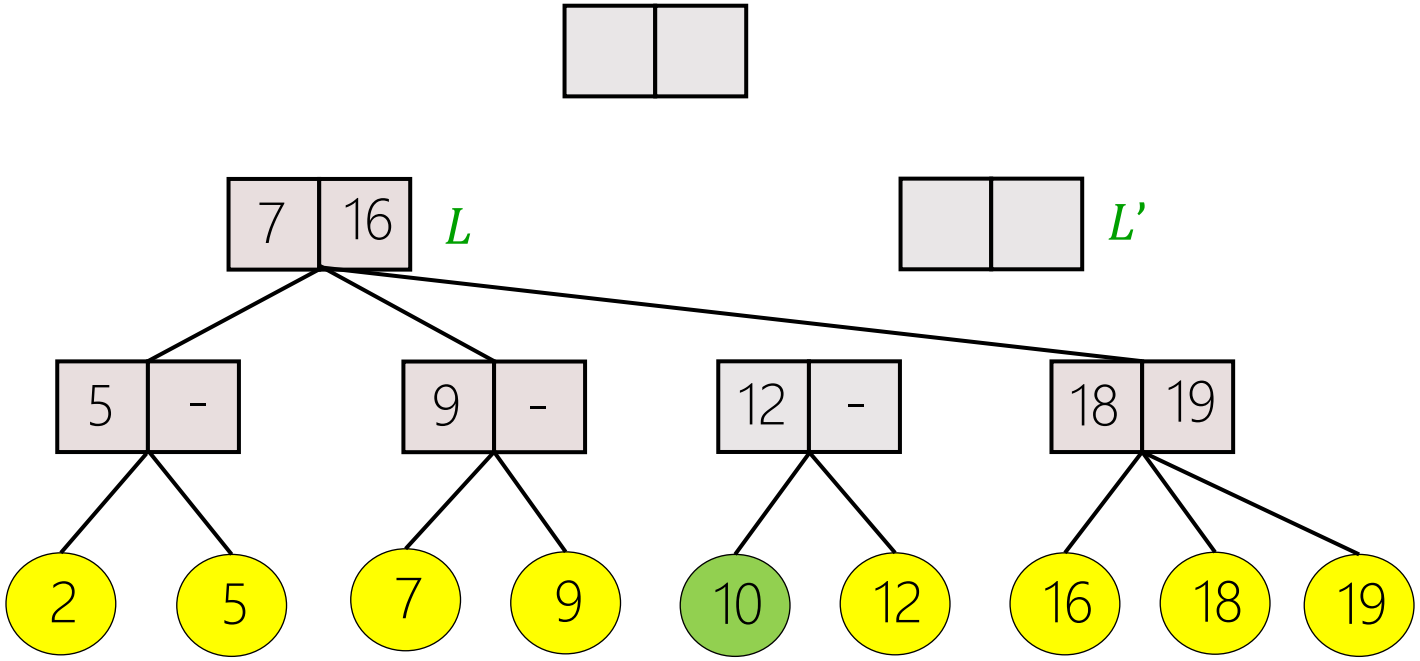
2-3 Tree: Insert - Example

- Insert a record with key value 10
  - Split-and-promote process again
    - Since L is the root, a new root has to be created as well



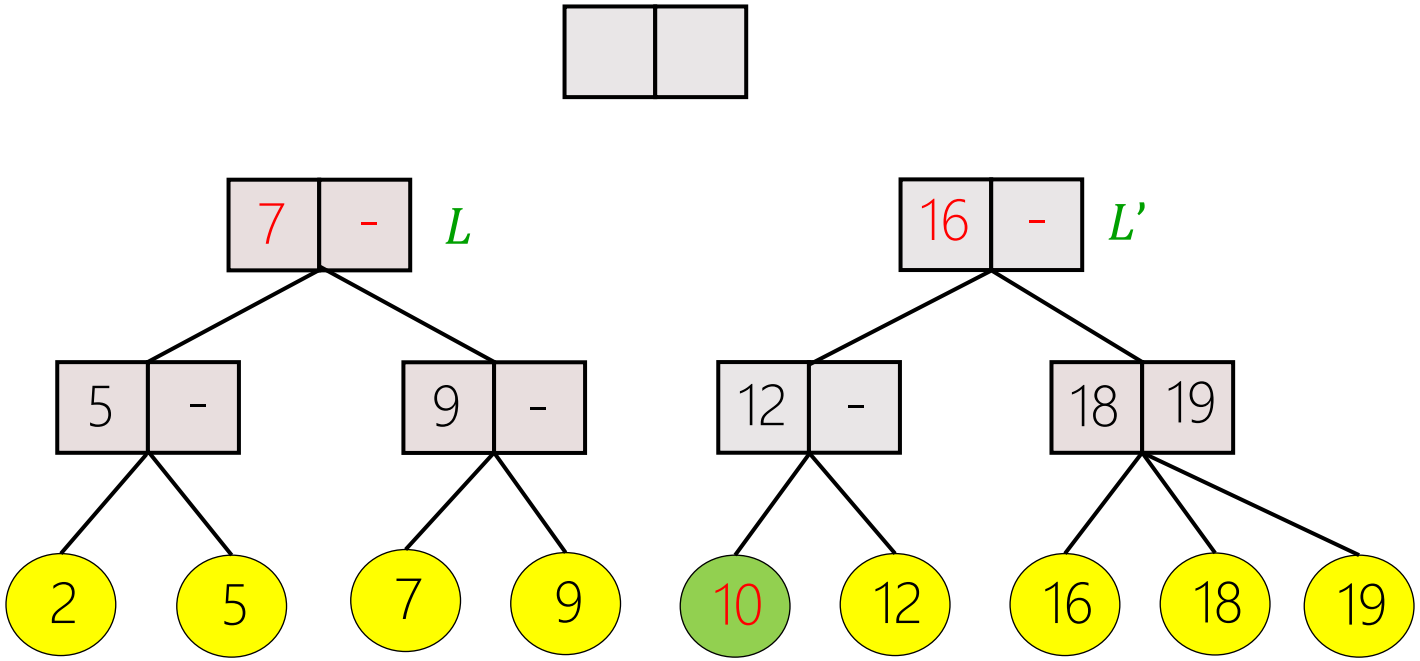
2-3 Tree: Insert - Example

- Insert a record with key value 10
  - Split-and-promote process again
    - Next rearrange the keys evenly among the two nodes
    - Update the key values in the parents



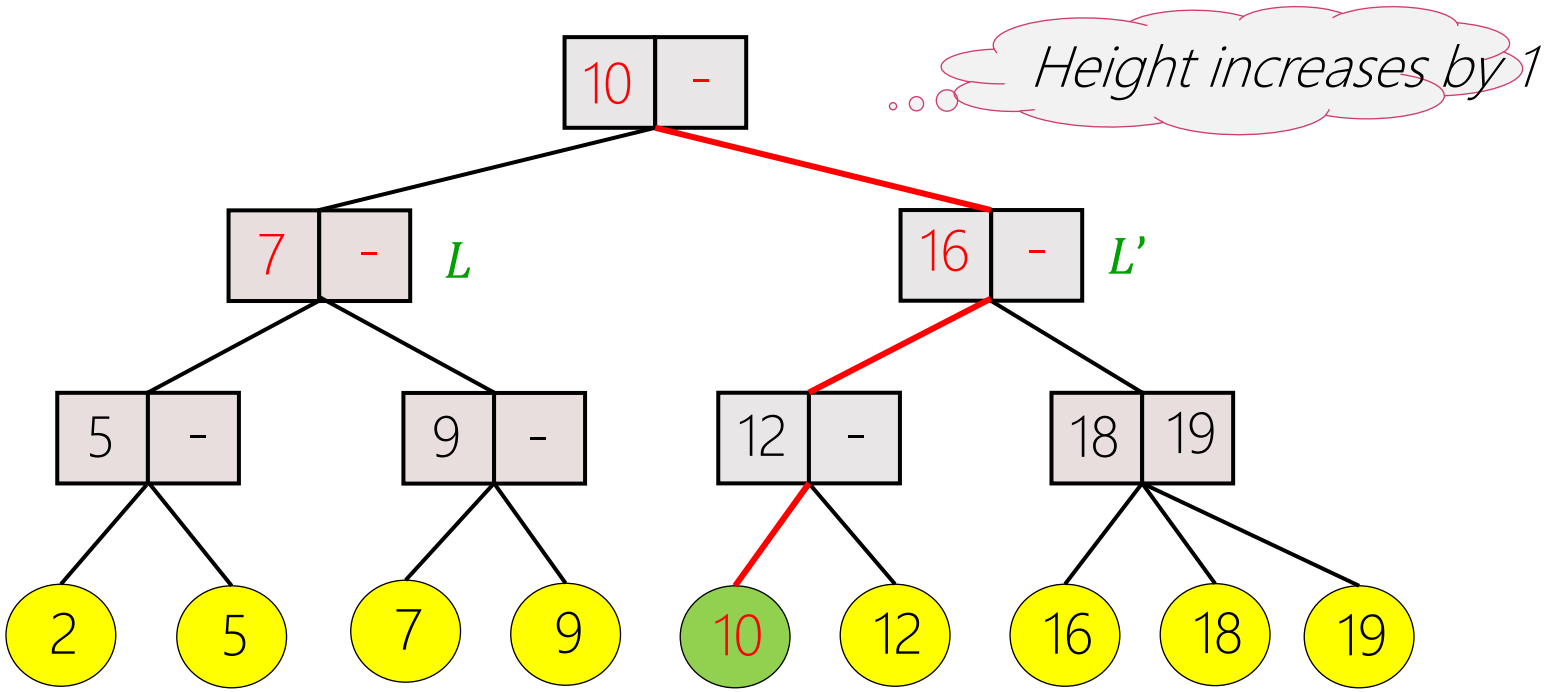
2-3 Tree: Insert - Example

- Insert a record with key value 10
  - Split-and-promote process again
    - Promote a copy of the least-valued key 10 in L'



2-3 Tree: Insert - Example

- Insert a record with key value 10
  - Split-and-promote process again
    - The insertion is complete



Quiz

- Does insertion order matter for 2-3 tree?
- Is the HEIGHT the same?
- What about BST?

2-3 Tree: Delete

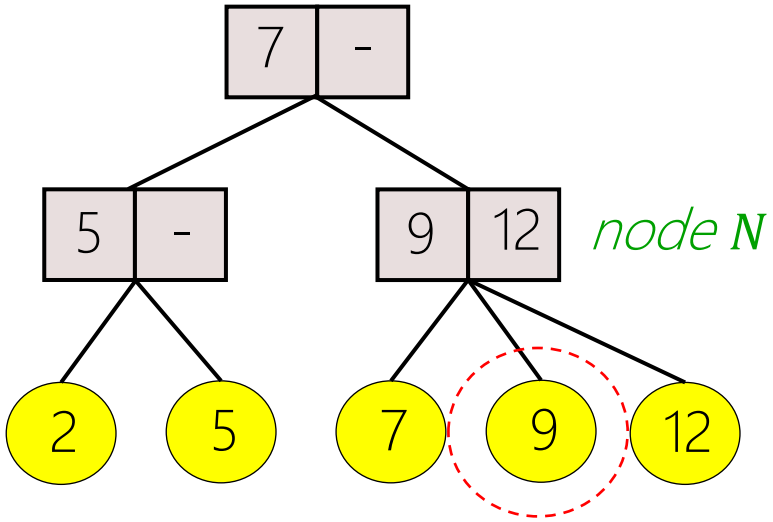
- Locate *node* N, one of whose children is *key* K to be deleted

- Three cases to consider:
  - If N has 3 children
    - Just remove K

  - If N has 2 children (Underflow)
    - Look at N's adjacent siblings to determine if they have a spare key that can be used to fill the gap
    - If not (i.e., neither sibling can lend a key to the underflow node N)
      - N must give its key to a sibling and be removed from the tree

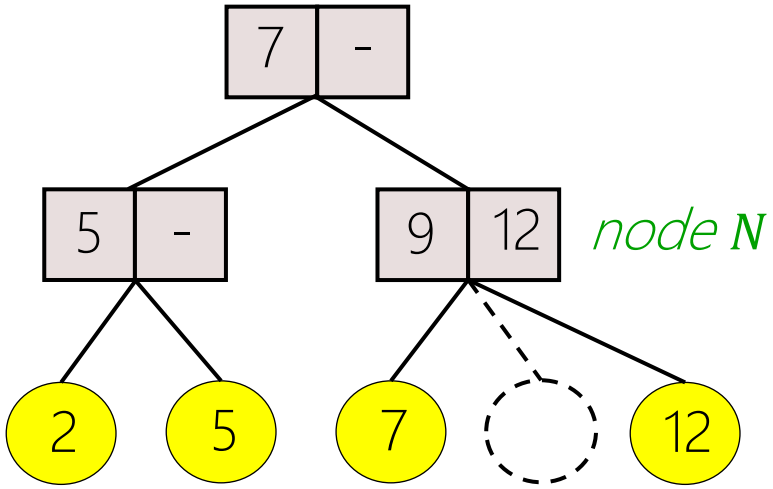
2-3 Tree: Delete (Case-1: Three Children)

- Delete a record with key value 9
  - Find the *node* $\boldsymbol{N}$, one of whose children is *key 9* to be deleted
  - Remove the record



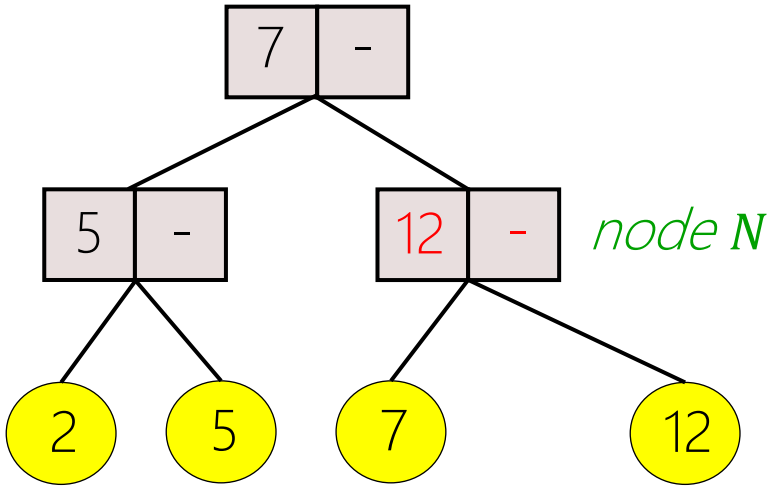
2-3 Tree: Delete (Case-1: Three Children)

- Delete a record with key value 9
  - Update the key value of the parent node N



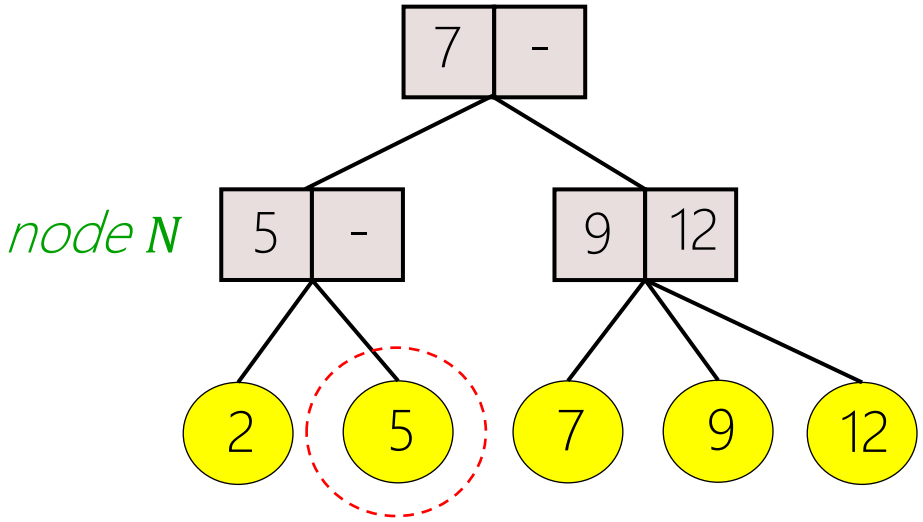
2-3 Tree: Delete (Case-1: Three Children)

- Delete a record with key value 9
  - Update the key value of the parent node N
  - The deletion is complete



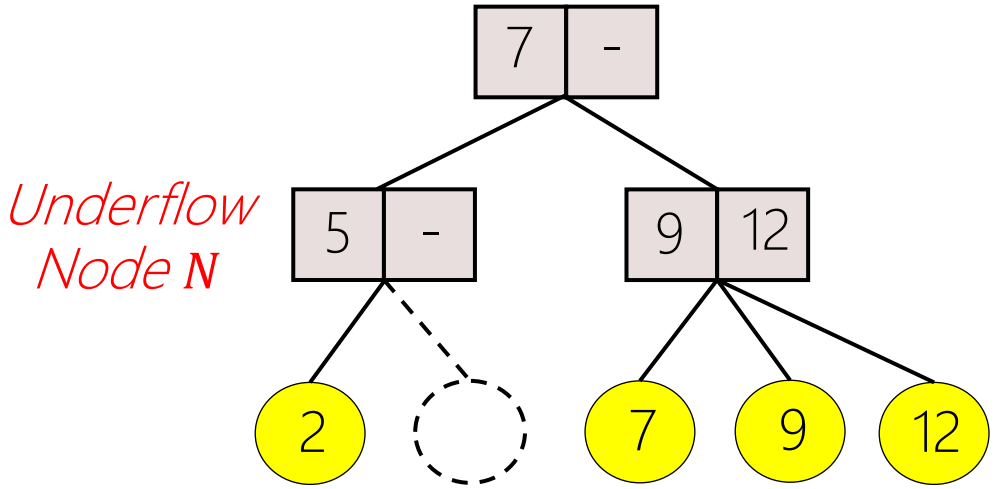
2-3 Tree: Delete (Case-2: Two Children)

- Delete a record with key value 5
  - Find the *node* $\boldsymbol{N}$, one of whose children is *key 5* to be deleted
  - Remove the record whose key is 5



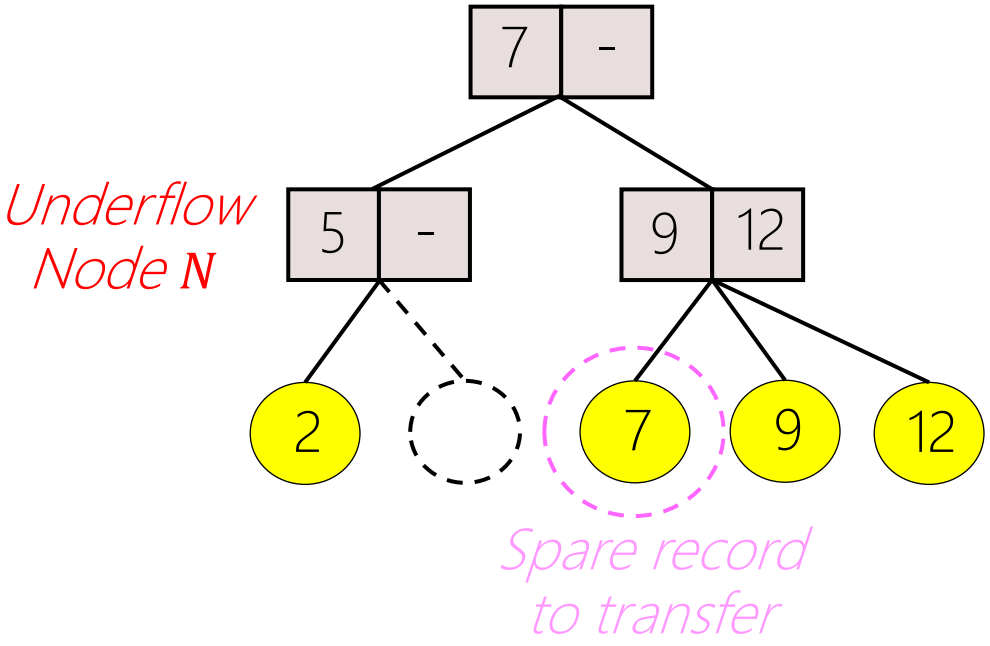
2-3 Tree: Delete (Case-2: Two Children)

- Delete a record with key value 5
  - Deleting a record causes the parent node N to underflow
  - Look for N's adjacent sibling that can lend a spare key to it



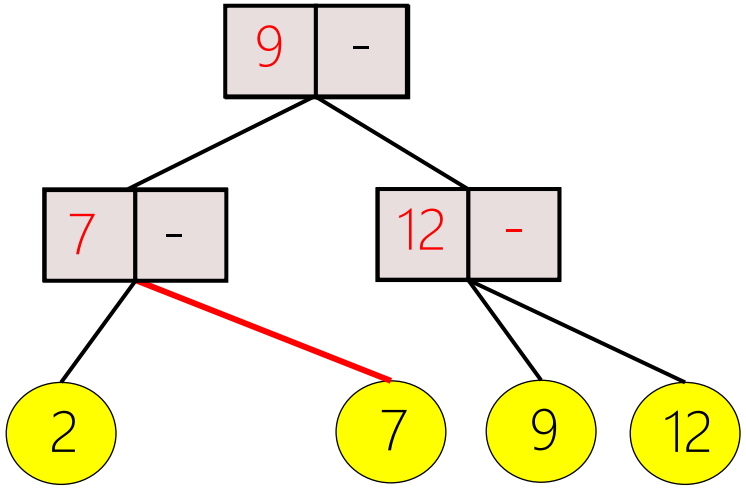
2-3 Tree: Delete (Case-2: Two Children)

- Delete a record with key value 5
  - Transfer the spare record to the underflow node N



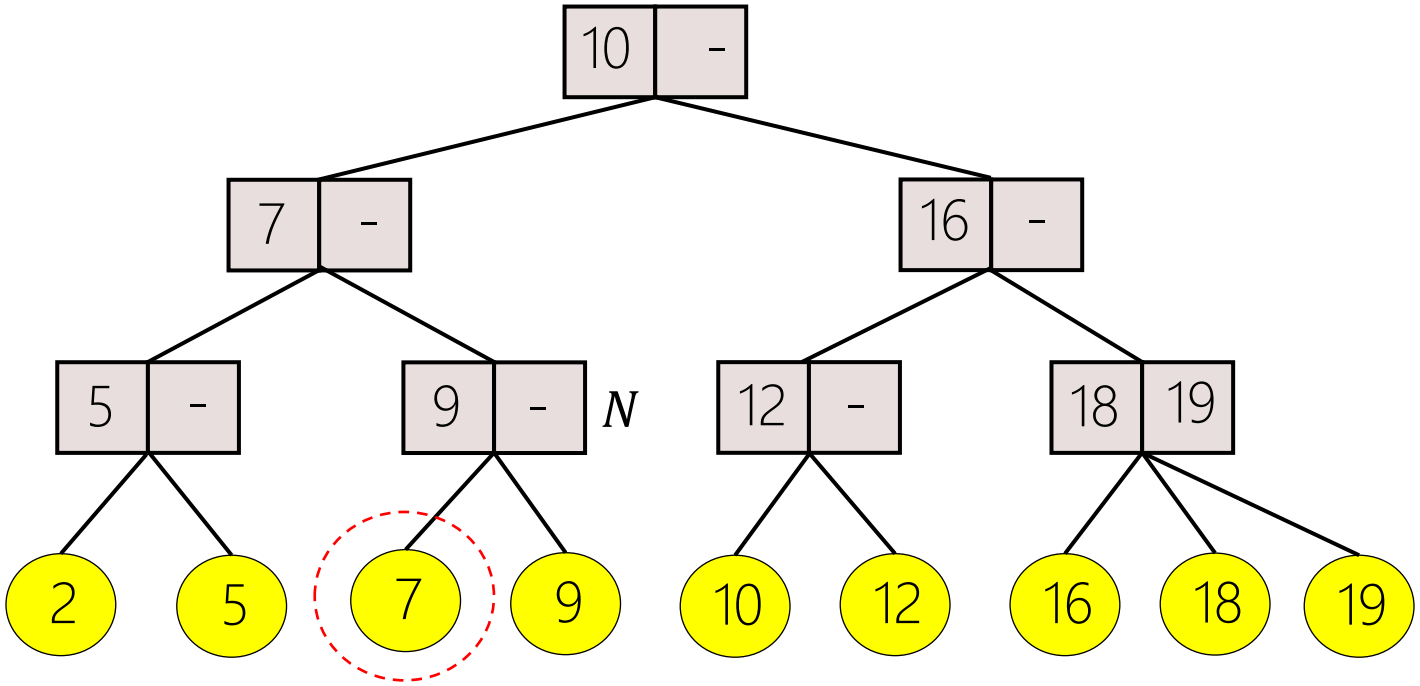
2-3 Tree: Delete (Case-2: Two Children)

- Delete a record with key value 5
  - Transfer the spare record to the underflow node N
  - Update the key value of the parent node
  - The deletion is complete



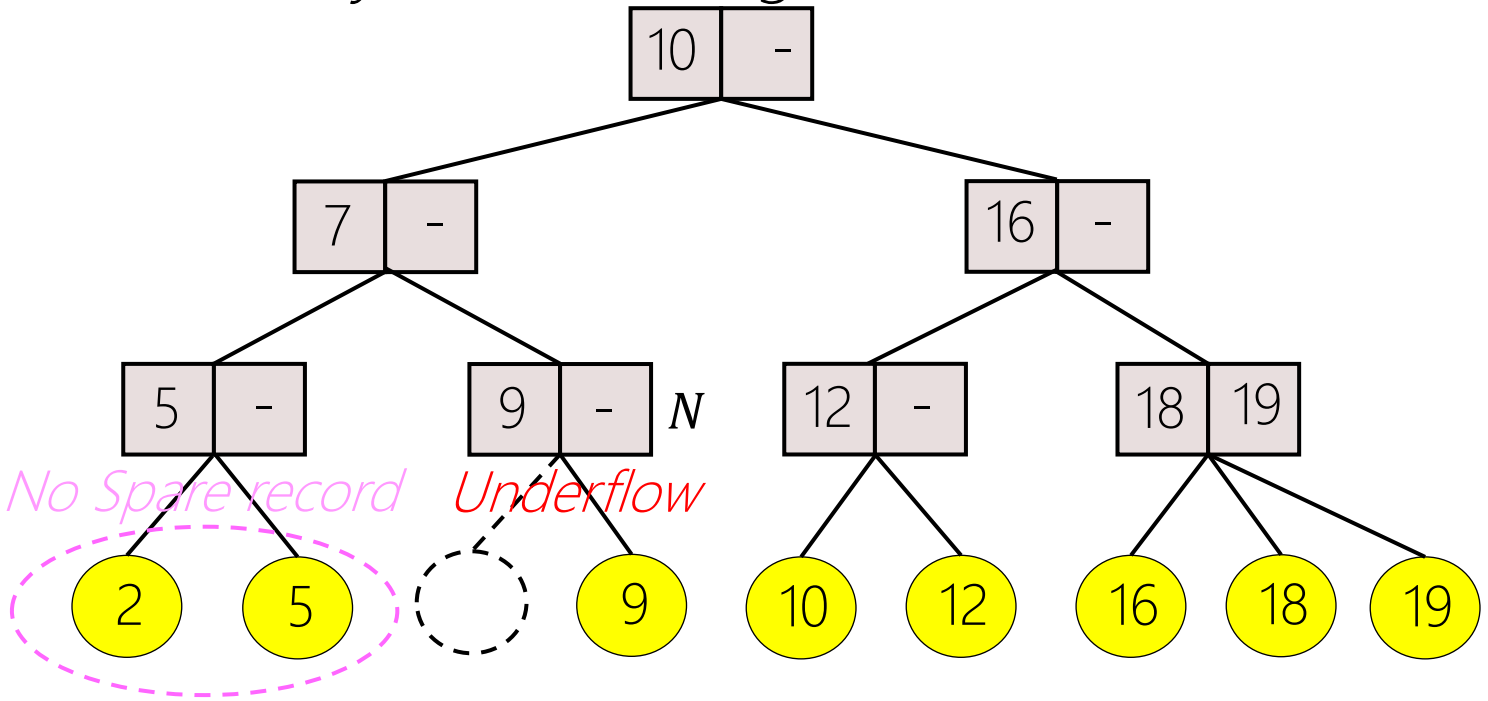
2-3 Tree: Delete (Case-3: No Siblings with a Spare Record)

- Delete a record with key value 7
  - Find the *node* $\boldsymbol{N}$, one of whose children is *key 7* to be deleted
  - Remove the record whose key is 7



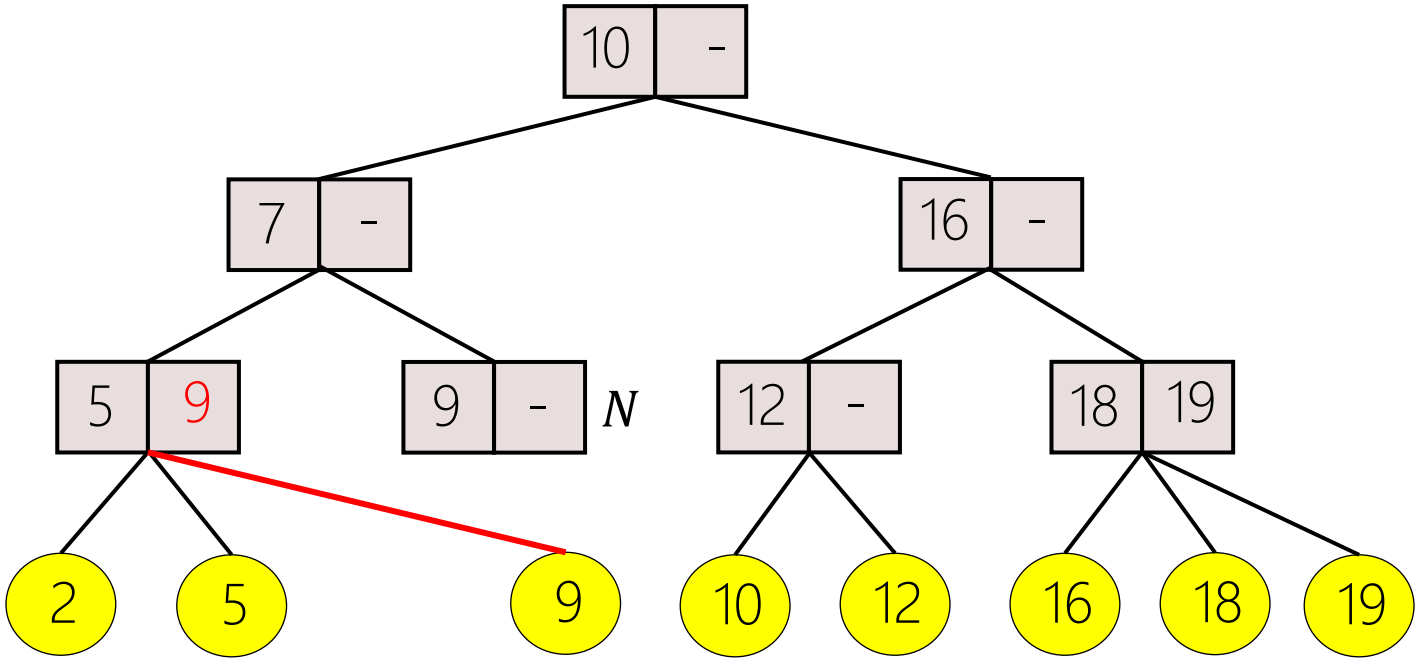
2-3 Tree: Delete (Case-3: No Siblings with a Spare Record)

- Delete a record with key value 7
  - Deleting a record causes the parent node N to underflow
  - No adjacent siblings that have a spare record
  - Transfer the record (*key 9*) to a sibling



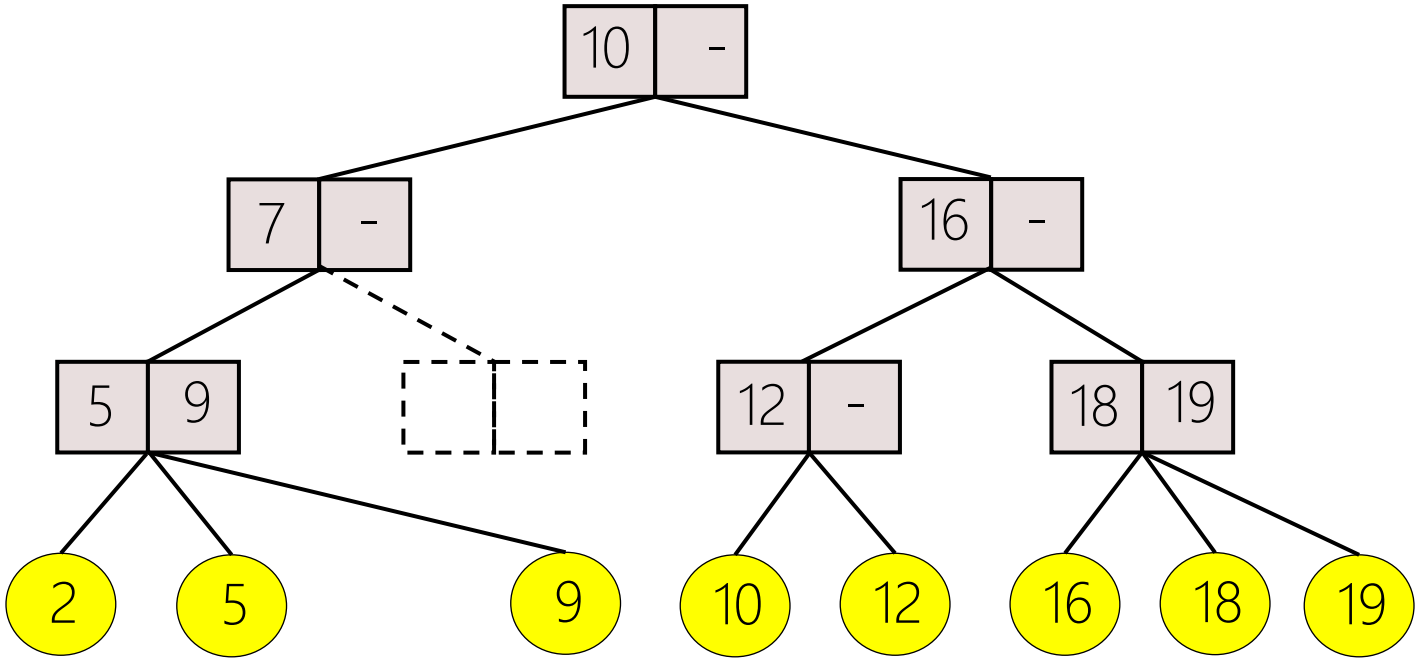
2-3 Tree: Delete (Case-3: No Siblings with a Spare Record)

- Delete a record with key value 7
  - Remove the parent node N recursively



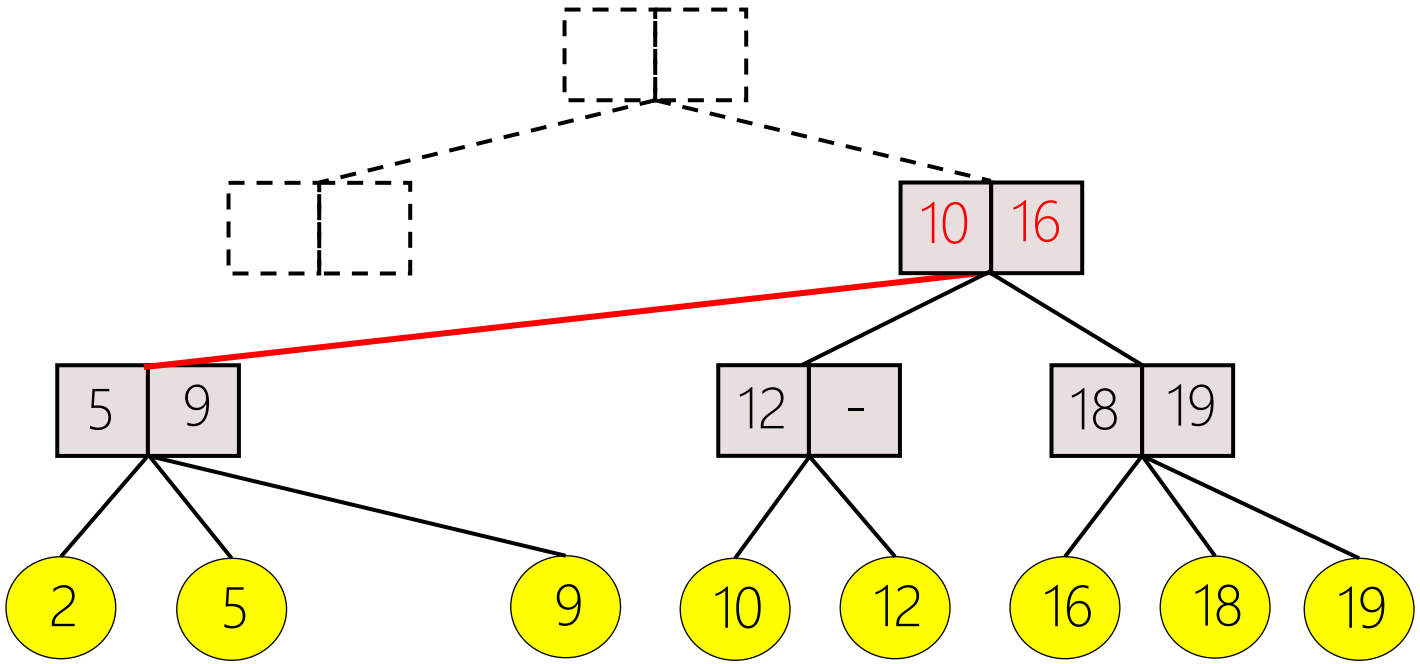
2-3 Tree: Delete (Case-3: No Siblings with a Spare Record)

- Delete a record with key value 7
  - Remove the parent node N recursively
    - Node-merge deletion process



2-3 Tree: Delete (Case-3: No Siblings with a Spare Record)

- Delete a record with key value 7
  - Remove the parent node N recursively
    - Node-merge deletion process



Quiz. Where does the tree grow and shrink?

Insertion grows the tree at the ROOT. Deletion shrinks it at the ROOT too.

- Why does this make rotations (AVL) unnecessary?

Comparison of B-trees with Sorted Arrays

- Disadvantages
  - More space overhead
  - More complex implementation

- Advantages
  - Updates are much easier
  - Search time is generally faster

Summary

- B-Tree / B+ Tree
- 2-3 Tree
- Perfectly Height-Balanced Tree
- Internal Nodes / Leaf Nodes
- Height Bound
- Search
- Insertion & Node Split
- Deletion (Three Children / Two Children / No Spare Siblings)
- Comparison with Sorted Arrays